Pepr Overview



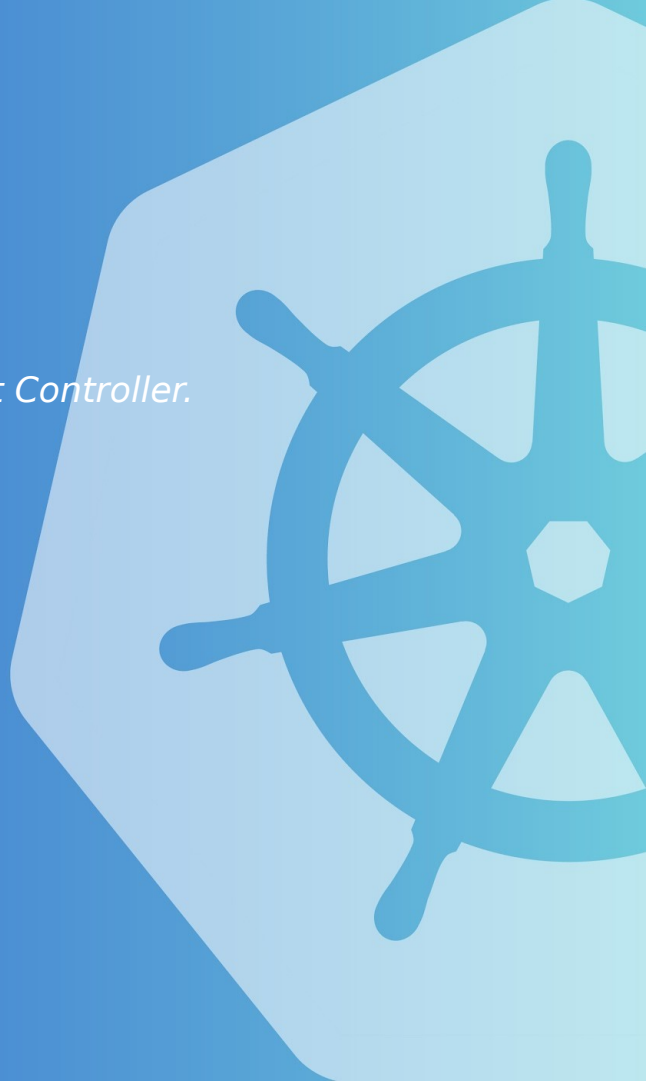
SREDay London

Building Smarter Kubernetes Workflows: Pepr for the modern SRE

kubernetes

What

Extremely free and flexible Kubernetes Admission, Watch, and Event Controller.



How

Secure by default, sane securityContexts, automation of complex configuration and deployments.

- *Fence app through Network Policy*
- *Generate/Configure KeyCloak, KeyCloak Client (AuthZ)*
- *Istio VirtualService and Gateway Configuration*

Configure your app right off the bat!



Why

Plain english expressiveness and the power of strongly typed, full featured programming language to react to events.



Background:

- The first commit was March 8th, 2023 by Jeff McCoy, CTO of Defense Unicorns who created the project and eventually created a team around the October timeframe
- The README of Pepr remains the same since it was first created, “Pepr is on a mission to save K8s from the tyranny of YAML, intimidating glue code, bash scripts, and other makeshift solutions” (McCoy, 2023).
- Zero Config Kubernetes Mutating and Validating Webhook and Controller generation
- It is tool created to automate legacy knowledge into code. Codify the knowledge of the Brents of the world wit TS ut needings to have a deep understanding of kubernetes/client-go
- It is written 💯 in 😳.
  - It has a fresh, easy to english-style API that just makes sense and it easy to understand and build upon
  - Pepr runs in cluster, really only must talk to kube-apiserver, latency/hops are minimal.
- Cloud Native



How to think about Pepr from a cloud native CNCF prospective



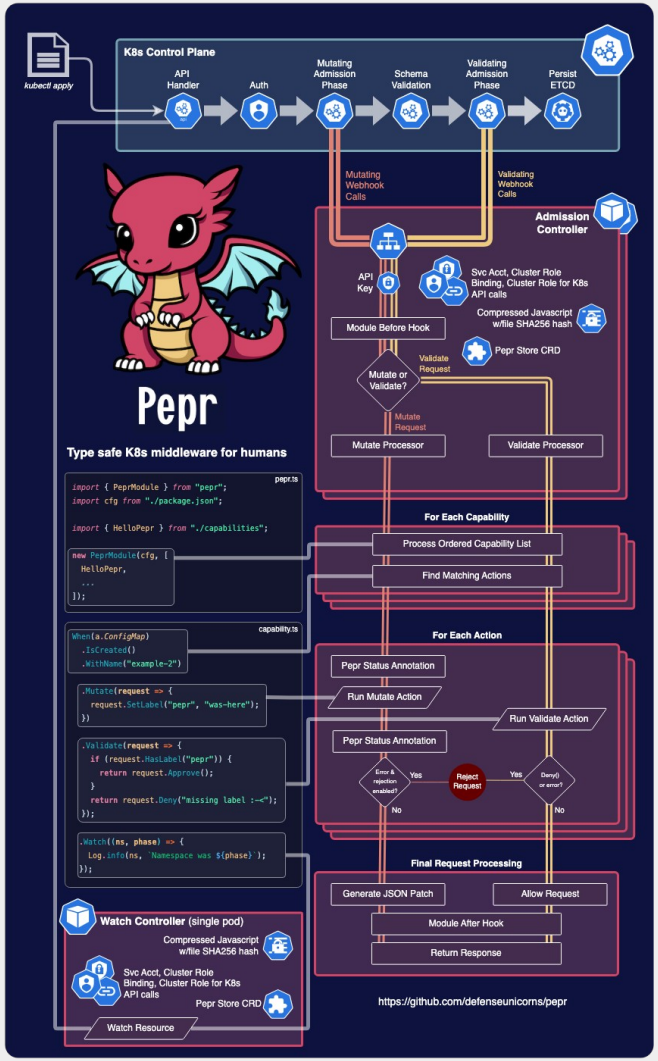
Pepr is a *framework*

- Creates Mutating/Validating Admission Webhooks
- Speaks Kubernetes “watch” (like any operator)
  - Has a custom Informer that caches, relists, and watches objects
- Understands CRDs
- Can reconcile execution on events based on a Kubernetes event in a Queue
- Speaks Server Side Apply to minimal network pressure
- Has a (custom) Kubernetes Client, KFC, consumed by Pepr as a library
- Can generate/interpret TypeScript classes from CRDs that are used in Pepr projects

Maybe, in my opinion, if you had to label it, a super rough, high level comparison is to say Pepr is a mix between something like a Kyverno and something like an Operator SDK.

The goals are obviously inherently different in each of these projects. Not really a 1:1 with anything that I am aware of.




kubectl apply
K8s Control Plane
API Handler
Auth
Mutating Admission Phase
Schema Validation
Validating Admission Phase
Persist ETCD
Mutating Webhook Calls
Validating Webhook Calls
Admission Controller
API Key
Svc Acct, Cluster Role Binding, Cluster Role for K8s API calls
Compressed Javascript w/file SHA256 hash
Module Before Hook
Pepr Store CRD
Mutate or Validate?
Validate Request
Mutate Request
Mutate Processor
Validate Processor
Pepr
Type safe K8s middleware for humans
pepr.ts
import { PeprModule } from "pepr";
import cfg from "./package.json";
import { HelloPepr } from "./capabilities";
new PeprModule(cfg, [
HelloPepr,
...
]);
capability.ts
When(a.ConfigMap)
.IsCreated()
.WithName("example-2")
.Mutate(request => {
request.SetLabel("pepr", "was-here");
})
.Validate(request => {
if (request.HasLabel("pepr")) {
return request.Approve();
}
return request.Deny("missing label :-<");
});
.Watch((ns, phase) => {
Log.info(ns, `Namespace was ${phase}`);
});
For Each Capability
Process Ordered Capability List
Find Matching Actions
For Each Action
Pepr Status Annotation
Run Mutate Action
Run Validate Action
Pepr Status Annotation
Error & rejection enabled?
Yes
No
Reject Request
Yes
Deny() or error?
No
Final Request Processing
Generate JSON Patch
Allow Request
Module After Hook
Return Response
Watch Controller (single pod)
Compressed Javascript w/file SHA256 hash
Svc Acct, Cluster Role Binding, Cluster Role for K8s API calls
Pepr Store CRD
Watch Resource
https://github.com/defenseunicorns/pepr

The API



Actions/Filters

```
When(ServiceMonitor)
.isCreatedOrUpdated()/IsCreated()/
isUpdated()/isDeleted()
.WithName(),InNamespace(),InRegexNamespace(),
WithLabel(),WithAnnotation(),WithDeletionTime
stamp(),WithNameRegex(),WithName()
.Alias
.Mutate()/Validate()
.Reconcile()/Watch
.Finalize()

**Executes callbacks on
Mutate,Validate,Reconcile,Watch,Finalize
```

When - Define Kubernetes resource to operate on

Event - Create/Delete/Update or CreateOrUpdate

Filters - Filter in Webhook and Watch URL and after event is passed

Mutate/Validate - Events from webhook

Reconcile/Watch - Watch events from Kubernetes API Server URL

Finalize - Creates a finalizer on the resources and removes after the callback



Pepr Store and Scheduler

Store
Provides ability to store, retrieve, update, delete values needed during admission or watch event processing

- Backed by ETCD through the `PeprStore` Resource
- Simple API
  - `setItem`
  - `getItem`
  - `setItemAndWait`
  - `removeItem`
  - `removeItemAndWait`
  - `clear()`
  - `subscribe`
  - `onReady`
- Backed by JSON Patch Operations to kube-apiserver to the `PeprStore` CR

OnSchedule
Enables the create of tasks that run once or on a schedule, eliminating the need to deploy separate `Jobs/CronJobs` or manage multiple codebases

- Uses separate instance of the `PeprStore` Resource to remain consistent if Pods are restarted
- Simple API
  - Name - scheduleName
  - Every - number of units
  - Unit - seconds, minutes, hours,
  - startTime
  - Run - the code to run on the schedule
  - Completions - how many times the code should run



```
OnSchedule({
  name: "refresh-aws-token",
  every: 23,
  unit: "hours",
  run: async () => {
    try {
      const token = await refreshAWSToken();
      Store.setItem("aws-token", token);
    } catch {
      Log.error(`Failed to refresh AWS token. ${new Date()}`);
    }
  },
})
```



```
/**
 * This policy ensures that Pods do not allow privilege escalation.
 *
 */
When(a.Pod)
  .IsCreatedOrUpdated()
  .Validate(request => {
    const violations = containers(request).filter(
      c =>
        c.securityContext?.allowPrivilegeEscalation === true ||
        c.securityContext?.privileged === true,
    );

    if (violations.length) {
      return request.Deny(
        securityContextMessage(
          "Privilege escalation is disallowed",
          ["allowPrivilegeEscalation = false", "privileged = false"],
          violations,
        ),
      );
    }

    return request.Approve();
  });
```



```
/**
 * Add sane CPU and Memory limits to containers
 *
 */
When(a.Pod)
  .IsCreatedOrUpdated()
  .Mutate(request => {
    containers(request).forEach(c => {
      // Ensure the resources field is defined
      c.resources = c.resources || {};

      if (c.resources.limits === undefined) {
        c.resources.limits = {
          cpu: "200m",
          memory: "256Mi",
        };
      }

      if (c.resources.requests === undefined) {
        c.resources.requests = {
          cpu: "100m",
          memory: "128Mi",
        };
      }
    });
  });
```



```ts
export async function networkPolicies(pkg: UDSPackage, namespace: string) {
  const customPolicies = pkg.spec?.network?.allow ?? [];
  const pkgName = pkg.metadata!.name!;

  // Get the current generation of the package
  const generation = (pkg.metadata?.generation ?? 0).toString();

  log.debug(pkg.metadata, `Generating NetworkPolicies for generation ${generation}`);

  // Create default policies
  const policies = [
    // All traffic must be explicitly allowed
    defaultDenyAll(namespace),

    // Allow DNS lookups
    allowEgressDNS(namespace),

    // Istio rules
    allowEgressIstiod(namespace),
    allowIngressSidecarMonitoring(namespace),
  ];

  // Process custom policies
  for (const policy of customPolicies) {
    const generatedPolicy = generate(namespace, policy);
    policies.push(generatedPolicy);
  }

  // Generate NetworkPolicies for any VirtualServices that are generated
  const exposeList = pkg.spec?.network?.expose ?? [];
  // Iterate over each exposed service, excluding directResponse services
  for (const expose of exposeList.filter(exp => !exp.advancedHTTP?.directResponse)) {
    const { gateway = Gateway.Tenant, port, selector = {}, targetPort } = expose;

    // Use the same port as the VirtualService if targetPort is not set
    const policyPort = targetPort ?? port;
```



```
 * Remove any remaining clients that are not in the refs list
 *
 * @param pkg the package to process
 * @param refs the list of client refs to keep
 */
export async function purgeSSOClients(pkg: UDSPackage, newClients: string[] = []) {
  // Check for any clients that are no longer in the package and remove them
  const currentClients = pkg.status?.ssoClients || [];
  const toRemove = currentClients.filter(client => !newClients.includes(client));
  for (const ref of toRemove) {
    const storeKey = `sso-client-${ref}`;
    const token = Store.getItem(storeKey);
    if (token) {
      await apiCall({ clientId: ref }, "DELETE", token);
      await Store.removeItemAndWait(storeKey);
    } else {
      log.warn(pkg.metadata, `Failed to remove client ${ref}, token not found`);
    }
  }
}
```



Automatic builds



Build Process

- Users create their module and build it which creates a new project and git repo
- Build is done through `npx pepr build`, which compiles their modules into a tiny secret that is consumed by the controller
- Pepr projects stay up to date through the Pepr dependency, ie DependaBot or Renovate
- Generates a helm chart, kubernetes manifest for kubectl apply, and a zarf file for air gap
- The consumers of Pepr only worry about creating their Pepr module
- Build options to scope down RBAC and several others



Closing

The Pepr API lowers the barrier of entry for whoever is maintaining a cluster without over simplification. It plays nice with other Admission Controllers and Operators giving the guard rails for compatibility without compromise.

